

Tennis Match Prediction

Project guide: what it is, how to run it, and what every part of the code does

Who this is for

Anyone receiving this project: a student, reviewer or developer who has never seen the code. No machine-learning background is needed to run it. The later chapters explain the internals for those who want to modify it.

Verification status

The full pipeline was run end to end on the real ATP data (1997 to January 2026, about 88,000 matches): feature building, training of logistic regression, gradient boosting and a linear SVM, prediction, the command-line tool and the web interface. The random forest was run on synthetic data only, because the test machine had a single CPU; it runs normally on a typical laptop. Real results are in chapter 9.

Contents

1. What this project is	3
2. Quick start	3
3. Project layout	4
4. Libraries used and why	5
5. The data	6
6. Code walkthrough	7
7. Feature reference	10
8. Outputs and saved files	11
9. Understanding the metrics	11
10. Design decisions worth knowing	12
11. Troubleshooting	12
12. Limitations and ideas for extension	13
13. Glossary	13

1. What this project is

This project predicts the winner of professional men's tennis (ATP) matches **before the match starts**, using only information that was available at that time: past results, form, surface records, head-to-head history, rankings and serve statistics. For any two players and a surface, it outputs each player's probability of winning.

It implements the approach of the Stanford CS229 report *Tennis Match Prediction using Machine Learning* (Ajay Krishna Amudan): a feature-heavy logistic regression compared against the simplest sensible benchmark, always predicting that the higher-ranked player wins (about 65% accurate in the report), plus random forest and SVM models. This implementation extends it with Elo ratings, gradient boosting, probability-quality metrics and a web interface.

What you get

- A **data pipeline** that downloads and cleans public ATP match files (the TML-Database).
- A **feature builder** that describes each match using only earlier matches (no look-ahead).
- A **training script** that fits four models, compares them with two baselines on a later season, and saves everything.
- A **web app** (Streamlit) and a **command-line tool** to ask 'who wins between player A and player B?'.

The problem in one paragraph

Each match has a winner and a loser, but a model needs examples of both outcomes. So every historical match is turned into two rows: one describing it from the winner's side (label 1) and one from the loser's side (label 0). The model learns which combinations of features go with winning. At prediction time both mirrored rows are scored and the results are averaged, so the answer never depends on which player is listed first.

2. Quick start

Requirements

- Python 3.9 or newer (developed and tested with Python 3.12, pandas 3.0, scikit-learn 1.8).
- Internet access on the first run, to download the yearly data files (about 20 MB in total).
- A few GB of free memory. Building features takes about a minute or two; training all four models takes several minutes.

Steps

```
# 1. Put the project folder somewhere and open a terminal inside it
python -m venv venv                # optional but recommended
source venv/bin/activate           # Windows: venv\Scripts\activate
pip install -r requirements.txt

# 2. Train (downloads data, builds features, trains models, saves to artifacts/)
python train.py

# 3. Launch the interface in your browser
streamlit run app.py
```

```
# Optional: predict from the terminal
python predict.py --a "Player One" --b "Player Two" --surface Clay --best-of 3
```

Step 2 only needs to be done once. It writes the **artifacts/** folder that the app and the command-line tool load. Re-run it to refresh the data or retrain.

Training options

Option	Default	Meaning
--start	1997	First season to load. Detailed serve statistics are only reliable from 1997 onward.
--end	current year	Last season to load.
--test-year	latest full season	Season used for testing. All earlier seasons are used for training. Use 2019 to mirror the paper.
--models	all four	Any of logreg random_forest gradient_boosting svm. Use --models logreg for a fast run.
--data-dir	data	Where downloaded CSV files and the cached feature table are stored.
--out	artifacts	Where trained models and player state are saved.
--refresh	off	Re-download the current and previous season (the source repository updates these).
--rebuild	off	Ignore the cached feature table and rebuild it (needed after changing features.py or the data).

If the automatic download fails

Open <https://github.com/Tennismylife/TML-Database> in a browser, download the yearly files (1997.csv to 2026.csv) and put them, unrenamed, in a folder called data/ inside the project. Run `python train.py` again; it uses local files first and only downloads years that are missing.

Important: the original dataset is gone

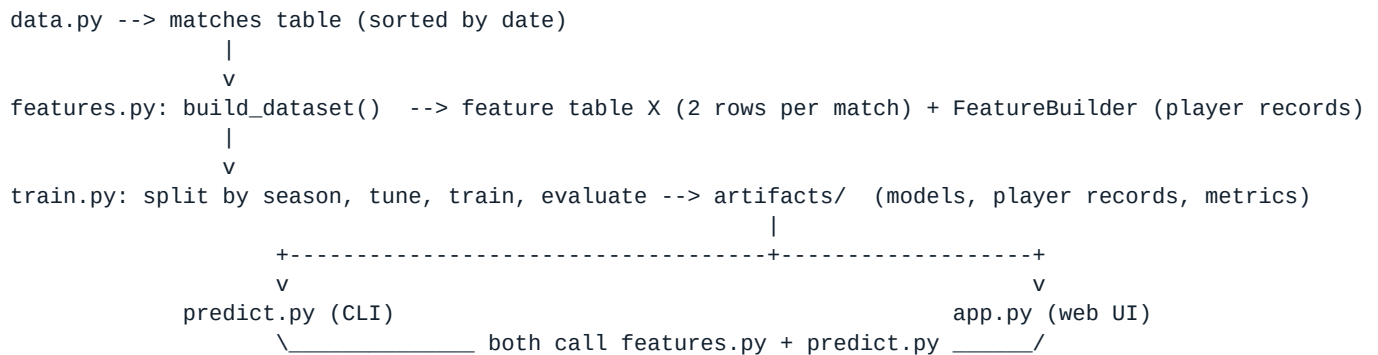
The paper and the first version of this project used Jeff Sackmann's `tennis_atp` repository. That repository was removed from GitHub (its links return 404), so the project now uses the TML-Database, which keeps the same column layout. Copies of Sackmann's old files, if you have them, still work: put them in data/ under their original names.

3. Project layout

File	Role
data.py	Downloads and cleans the raw match files. Returns one chronologically sorted table of matches.
features.py	The core of the project. Keeps a running record for every player and converts each match into model-ready numbers. Used identically by training and by the app.

File	Role
train.py	Runs everything: loads data, builds features, splits by time, tunes, trains four models, evaluates, saves artifacts.
predict.py	Loads saved artifacts and answers 'who wins?' for any pair of players. Also builds the side-by-side comparison and the explanation. Has a command-line entry point.
app.py	The Streamlit web interface (Predict, Model results and How it works tabs).
requirements.txt	List of Python packages to install.
.streamlit/config.toml	Fixes the app to a light theme with the project's colour palette.
README.md	Short version of this guide.
data/ (created)	Downloaded CSV files and the cached feature table (features_START_END.pkl).
artifacts/ (created)	models.joblib, state.joblib, features.json, metrics.json.

How the pieces connect



4. Libraries used and why

Library	Used in	Why it is needed
pandas	data.py, features.py, train.py, predict.py, app.py	Reads the CSV files, cleans and sorts them, parses dates, and holds the feature table as a DataFrame that scikit-learn accepts. Also formats the results table in the app.
numpy	features.py, train.py	Fast numeric arrays and NaN handling for masks, slicing by season, log-loss and Brier calculations.
scikit-learn	train.py, predict.py	All machine learning: LogisticRegression, RandomForestClassifier, HistGradientBoostingClassifier, LinearSVC with CalibratedClassifierCV, SimpleImputer (fills missing values), StandardScaler (puts features on one scale) and Pipeline (chains these steps so training and prediction stay consistent).
joblib	train.py, predict.py	Saves and loads trained models, the player records, and the cached feature table. Handles large numeric objects efficiently.
streamlit	app.py	Turns a Python script into a web interface with dropdowns, tabs and tables, with no HTML or JavaScript project needed.

Library	Used in	Why it is needed
Standard library	all files	<code>argparse</code> (command-line options), <code>collections.deque</code> (rolling windows of recent results), <code>urllib</code> (download), <code>json</code> , <code>pathlib</code> , <code>re</code> , <code>math</code> , <code>html</code> (safe text in the app), <code>datetime</code> .

Why these models

Logistic regression is the paper's main model: fast, well calibrated, and its coefficients can be inspected, which powers the 'What moved the estimate' panel. **Random forest** and **gradient boosting** capture non-linear effects and interactions. **Linear SVM** replaces the paper's kernel SVM, which was reported as extremely slow to train; it is wrapped in a calibrator so it outputs probabilities.

5. The data

Source: the TML-Database (github.com/Tennismylife/TML-Database), a maintained ATP results database that was originally inspired by Jeff Sackmann's `tennis_atp` files and keeps their column layout. It has one file per season, `YYYY.csv`, with one row per match (Grand Slams, Masters, ATP 500/250 events, Tour Finals, Davis Cup).

- **No Challenger or qualifying-only files.** The paper added Challenger matches (from 2010) so newer players would have a history. The TML-Database does not include them. If you have Sackmann-style `atp_matches_qual_chall_YYYY.csv` files, put them in `data/` and they are used automatically (training rows only; testing uses main-tour matches).
- **Player ids are text** (for example T786), not numbers. The code treats ids as strings.
- **The GitHub copy can lag behind.** When this guide was written, `2026.csv` held only matches up to 17 January 2026. The app shows a notice when the data is more than 60 days old. The live database is on stats.tennismylife.org; if you export a newer season from there in the same column layout, save it as `2026.csv` in `data/`.

Columns the code reads

Group	Columns
Match context	<code>tourney_id</code> , <code>tourney_name</code> , <code>tourney_date</code> , <code>tourney_level</code> , <code>surface</code> , <code>round</code> , <code>match_num</code> , <code>best_of</code> , <code>score</code>
Players (prefix <code>winner_</code> or <code>loser_</code>)	<code>id</code> , <code>name</code> , <code>hand</code> , <code>ht</code> (height), <code>age</code> , <code>rank</code> , <code>rank_points</code> , <code>seed</code>
Serve statistics (prefix <code>w_</code> or <code>l_</code>)	<code>ace</code> , <code>df</code> (double faults), <code>svpt</code> (serve points), <code>1stIn</code> , <code>1stWon</code> , <code>2ndWon</code> , <code>SvGms</code> (service games), <code>bpSaved</code> , <code>bpFaced</code>

Cleaning steps (`data.py`)

- Drop rows missing a winner id, loser id, surface or date; convert ids to text.
- Keep only Hard, Clay, Grass and Carpet surfaces (capitalisation is normalised).
- Remove walkovers and defaults marked W/O or DEF (no match was really played).
- Convert the date, and create day (days since 1970, used for fast '30 days ago' checks) and year.

- Sort by day, tournament, round order (Q1 < R32 < R16 < QF < SF < F) and match number. A player's earlier-round result must be recorded before their next round is predicted.
- Missing serve statistics are not dropped: those matches simply do not update a player's serve averages. Other missing values are filled with the median at training time.

Licence and terms

The TML-Database is for educational, analytical and research use; redistribution or commercial use of the raw data without permission is not allowed. It builds on Jeff Sackmann's data (CC BY-NC-SA 4.0). Keep use non-commercial, credit both sources, and do not republish the CSV files.

6. Code walkthrough

6.1 data.py

Item	What it does
SOURCE	URL prefix of the yearly files in the TML-Database repository.
ROUND_ORDER	Maps round names to a sortable number so matches are ordered correctly within a tournament.
FILE_RE	Pattern that accepts YYYY.csv (TML) and Sackmann-style atp_matches_YYYY.csv / atp_matches_qual_chall_YYYY.csv, and ignores everything else (for example the player list).
download(data_dir, start, end, refresh)	For each season it fetches YYYY.csv into data_dir. Seasons already present are skipped (except the last two with refresh). A failed download is reported and skipped rather than crashing.
load_matches(data_dir, start, end)	Reads every matching CSV, tags each row as atp or chall, applies the cleaning steps in chapter 5 and returns one sorted DataFrame. If nothing is found it stops with manual download instructions.

6.2 features.py

This file is the heart of the project. Its guiding rule: **when computing features for a match, use only what was known before that match.** It achieves this by walking through matches in date order, reading each player's record, and only *then* updating the records with the result.

Helpers

Function	Purpose
_num(x, default)	Safely converts a value to a float; returns the default for blanks or NaN.
_ratio(n, d)	Returns n/d, or NaN when the denominator is zero (for example a player with no recorded serve points). NaN is filled later.
attrs_from_row(r, side)	Extracts a player's attributes as known before the match: rank (minimum 1, unranked = 1500), log of rank, ranking points (default 0), seed (default 40 = unseeded), height, age and a left-handed flag.
match_ctx(r)	Packages match context: surface, tournament name, day, and whether it is best-of-five.

class PlayerState: one player's running record

Created once per player and updated after each match. It stores: overall wins and losses; wins/losses per surface, per tournament, per opponent, and per (opponent, surface); the last 25 results overall and per surface (a deque that automatically discards the oldest); results from the last 30 days; overall and per-surface Elo rating; cumulative serve totals; the latest known attributes; and the date last played.

class FeatureBuilder: turns records into model inputs

Method	What it does
player_feats(s, ctx)	Computes the ~37 per-player numbers for one player at the moment of the match (win rates, form windows, serve rates, Elo, and so on; full list in chapter 7).
common(A, B)	Finds opponents both players have faced and sums each player's wins and losses against them. This is the paper's 'common opponents' feature and helps when two players have never met.
_row(...)	Builds one feature row 'from player X's point of view': most features are X minus Y differences; rank, ranking points, seed, height and handedness are kept as separate raw values for each player (the paper found differences hide that rank 1 vs 6 is not like 200 vs 206). Adds head-to-head, common-opponent, surface and best-of-five fields.
pair_rows(a, b, ctx, ia, ib)	Returns two mirrored rows: <i>a</i> as player 1 and <i>b</i> as player 1. Used for both training and prediction.
update(r, ctx, aw, al)	After a match, updates both players: Elo (overall and surface), wins/losses in every category, recent-results windows, 30-day window, serve totals (only when all serve columns are present), latest attributes, names, and the list of tournaments.

build_dataset(matches)

Loops over all matches in order. For each, it computes the two mirrored rows *before* calling `update`, labels the winner-first row 1 and the loser-first row 0, and records the match id (`mid`). It returns the full feature table plus the final `FeatureBuilder`, whose player records reflect every match in the data. The app uses those final records to predict upcoming matches.

Elo in this project

Every player starts at 1500. After a match the winner gains and the loser loses points in proportion to how surprising the result was (expected win probability = $1 / (1 + 10^{(\text{opponent minus own})/400})$). The step size is $K = 250 / (\text{matches played} + 5)^{0.4}$, so new players' ratings move quickly and veterans' move slowly. A separate Elo per surface captures clay or grass specialists.

6.3 train.py

Function / step	What it does
make_model(name, c)	Returns a scikit-learn Pipeline. Logistic regression: median imputer, scaler, L2 logistic regression with the liblinear solver. Random forest: 300 trees, entropy criterion, minimum 10 samples per leaf. Gradient boosting: depth 4, learning rate 0.05, 300 iterations, L2 penalty. SVM: linear SVM wrapped in a 3-fold probability calibrator.

Function / step	What it does
<code>win_probs(model, x)</code>	Scores both mirrored rows and returns, per match, the probability that the actual winner wins: $0.5 * (P(\text{winner-first row}) + 1 - P(\text{loser-first row}))$.
<code>score(p_win)</code>	Accuracy (share of matches with probability above 0.5), log loss and Brier score.
<code>pick_test_year(matches)</code>	Chooses the most recent completed season (before the current calendar year) with at least 1500 main-tour matches, so a half-finished season is not the test set.
<code>main()</code>	The orchestrator; see the steps below.

What `main()` does, in order

- **Load:** download if needed, read and clean matches, choose the test season.
- **Features:** build the feature table (or load the cached copy `data/features_START_END.pkl`; use `--rebuild` to force a rebuild).
- **Split by time:** train on all rows from seasons *before* the test year (including any Challenger files you added); test only on ATP main-tour matches of the test year. Random splitting is deliberately avoided because it would leak future information.
- **Tune:** for logistic regression, try `C` in `{0.003, 0.01, 0.03, 0.1, 1.0}` by training on seasons before (test year - 1) and picking the lowest log loss on the season (test year - 1). The test season is never used for tuning.
- **Baselines:** compute the accuracy of 'higher-ranked player wins' (ties and unranked players excluded) and 'higher Elo wins' on the same test matches.
- **Train and evaluate:** fit each requested model on the training rows and score it on the test matches.
- **Save:** write the four files in `artifacts/` (chapter 8).

6.4 predict.py

Function	What it does
<code>load_artifacts(model_dir)</code>	Loads models, player records, feature list and metrics from the artifacts folder.
<code>active_players(fb)</code>	Lists players with at least 20 matches who played within the last 730 days of the data, best rank first. Used to fill the app's dropdowns.
<code>predict_match(art, a, b, surface, best5, tournament, model)</code>	Builds the two mirrored rows for players <code>a</code> and <code>b</code> as of the last date in the data, scores them with the chosen model, and averages to give $P(a \text{ wins})$. For logistic regression it also returns the six features with the largest push on the result (coefficient times standardised value).
<code>snapshot(art, a, b, surface)</code>	Builds the side-by-side facts table: ranking, Elo (overall and on the surface), career record, surface record, last 10 matches, last 30 days, head-to-head (overall and surface) and record against common opponents.
<code>pretty(name)</code>	Converts internal feature names like <code>d_form10_w</code> to readable text such as 'Wins in last 10 matches overall'.
<code>__main__</code> block	Command-line tool: finds players by name (exact match first, then partial), predicts, and prints both probabilities.

Prediction reference date is the **last match date in the data**, so 'last 30 days' means the 30 days before the data ends, not before today. A player's rank and ranking points are as of their most recent recorded match.

6.5 app.py

- **Loading:** `st.cache_resource` loads the artifacts once, not on every click. If they are missing, the page shows an instruction to run `python train.py`.
- **Predict tab:** choose Player A, Player B, surface, best of 3 or 5, an optional tournament (adds each player's history at that event) and a model. The result shows each player's win probability, 'fair odds' (1 divided by the probability, before any bookmaker margin), and a bar split at the predicted share.
- **Match-up table:** the output of `snapshot`, so users can sanity-check the prediction against plain facts.
- **What moved the estimate:** shown for logistic regression only; bars extending right favour Player A, left favour Player B.
- **Model results tab:** the saved metrics table with both baselines, so users see whether the models actually beat rankings.
- **How it works tab:** a plain-language summary of data, features and method.
- **Styling:** custom CSS (Manrope font, court-blue accent, tennis-ball-yellow divider on the probability bar). All player names are HTML-escaped before display.

7. Feature reference

There are about 60 columns per row. Names beginning with `d_` are 'Player 1 minus Player 2'.

Feature(s)	Meaning	Paper category
<code>d_wins</code> , <code>d_losses</code> , <code>d_win_pct</code>	Career wins, losses and win rate	Overall performance
<code>d_surf_wins</code> , <code>d_surf_losses</code> , <code>d_surf_win_pct</code>	Same, on the match's surface	Surface level overall form
<code>d_tour_wins</code> , <code>d_tour_losses</code>	Record at this specific tournament	Tournament form
<code>d_form5/10/15/25_w</code> and <code>_l</code>	Wins and losses in the last 5, 10, 15, 25 matches	Match-based form
<code>d_sform5/10/15/25_w</code> and <code>_l</code>	Same, counting only matches on this surface	Surface match-based form
<code>d_month_w</code> , <code>d_month_l</code>	Wins and losses in the 30 days before the match (a proxy for injury or rust)	Time-based form
<code>h2h_diff</code> , <code>h2h_n</code>	Player 1's wins minus losses against Player 2, and number of meetings	Overall head to head
<code>h2h_surf_diff</code>	Same, on this surface	Surface head to head
<code>co_w_diff</code> , <code>co_l_diff</code> , <code>co_n</code>	Wins/losses differences against common opponents and how many there are	Common opponents
<code>p1_/p2_ rank</code> , <code>log_rank</code> , <code>rank_pts</code> , <code>seed</code> , <code>ht</code> , <code>left</code>	Raw (not differenced) ranking, log-rank, ranking points, seed, height, left-handed flag for each player	Player attributes
<code>d_age</code>	Age difference	Extra

Feature(s)	Meaning	Paper category
d_ace_rate, d_df_rate, d_first_in, d_first_won, d_second_won, d_bp_saved	Career serve rates: aces, double faults, first serves in, first- and second-serve points won, break points saved	Historic average match statistics
d_sv_games, d_aces_total, d_dfs_total	Cumulative service games, aces, double faults (experience)	Cumulative match statistics
d_elo, d_elo_surf	Elo rating overall and on this surface	Extra (not in the paper)
best5, surf_hard, surf_clay, surf_grass	Match format and surface flags	Context

8. Outputs and saved files

File	Contents	Used by
artifacts/models.joblib	Dictionary of fitted models by name	predict.py, app.py
artifacts/state.joblib	The FeatureBuilder with every player's final record (needed to compute features for new match-ups)	predict.py, app.py
artifacts/features.json	Exact column order the models expect	predict.py
artifacts/metrics.json	Test season, number of matches, baselines, tuned C, and each model's accuracy, log loss and Brier score	app.py
data/features_START_END.pkl	Cache of the feature table and FeatureBuilder, to skip the slowest step on re-runs	train.py

Handling saved files

Files are stored with joblib (pickle). Only load artifacts you created or trust, and use the same package versions that created them; otherwise re-run `python train.py`.

9. Understanding the metrics

Metric	How to read it
Accuracy	Share of test matches where the model gave the eventual winner more than 50%. The paper reports 71% for logistic regression against a 65.5% ranking baseline. Roughly high-60s to low-70s is realistic for this problem.
Log loss	Penalises confident wrong predictions heavily. Lower is better. It rewards well-calibrated probabilities, which matter most if you compare against betting odds.
Brier score	Average squared error of the probability. Lower is better; always saying 50% scores 0.25.
Baselines	'Higher-ranked wins' and 'higher Elo wins'. A model is only useful if it beats these.

Results on the real data

Measured with the TML-Database (1997 to January 2026). Test matches are ATP main-tour matches of the test season; the same matches are used for every row.

Test season	Model	Accuracy	Log loss	Brier
2025 (2,839 matches)	Higher-ranked wins	64.3%	-	-
	Higher Elo wins	63.8%	-	-
	Logistic regression	65.8%	0.620	0.216
	Gradient boosting	65.6%	0.616	0.215
	Linear SVM	65.7%	0.621	0.216
2019, the paper's split (2,700 matches)	Higher-ranked wins	61.4%	-	-
	Higher Elo wins	64.2%	-	-
	Logistic regression	66.3%	0.609	0.212
	Gradient boosting	65.8%	0.610	0.212

How to read this. The models beat the ranking baseline (by about 1.5 points in 2025 and about 5 points in 2019) and give sensible probabilities, but they do not reach the paper's reported 71%. Two things differ from the paper: the ranking baseline here is 61 to 64% rather than 65.5% (this dataset's ranking columns or match mix differ from the original), and Challenger history is not available. Elo alone is a strong benchmark, and the extra features add only a small gain over it. Treat 65 to 67% as the realistic level for this data.

Bookmaker odds are very efficient, so consistently beating them is difficult. Treat outputs as estimates, and do not treat this project as betting advice.

10. Design decisions worth knowing

- **No data leakage.** Features come from records that exclude the match being predicted, and all splits are by time.
- **One code path.** The app calls the same `pair_rows` used in training, so what the model sees live matches what it saw in training.
- **Mirrored rows.** Prevent the model from learning 'the winner is always in the first column' and make predictions independent of player order.
- **Raw ranks plus differences.** Following the paper, rank, points and height are included per player rather than only as differences.
- **Optional Challenger data.** If Challenger files are supplied they add history for newer players; only main-tour matches are used for testing.
- **Median imputation.** Missing values (for example height or serve statistics) are filled with the training median instead of discarding matches.

11. Troubleshooting

Symptom	Cause and fix
'No match files found' when training	The automatic download failed or was blocked. Download the yearly CSV files from the TML-Database page and put them in data/ as described in chapter 2.
App says no trained model found	Run <code>python train.py</code> in the project folder first so <code>artifacts/</code> exists.
Training is slow	The first run builds features for about 88,000 matches (a minute or two) and trains four models; the random forest is the slowest. Later runs reuse the cache. Use <code>--models logreg</code> for speed.
Out of memory	Reduce the range, for example <code>--start 2005</code> , or train fewer models.
Errors after editing features.py	The cache is stale. Run <code>python train.py --rebuild</code> .
Unpickling or version errors in the app	Package versions differ from those used in training. Retrain in the current environment.
The app says the data ends in January 2026	The GitHub copy of the current season is out of date. Add a newer 2026.csv (see chapter 5) and run <code>python train.py --refresh --rebuild</code> .
Player not in the dropdown	Only players with 20+ matches who played within two years of the data end are listed. Adjust <code>active_players</code> in <code>predict.py</code> .

12. Limitations and ideas for extension

Known limitations

- The data currently stops in mid-January 2026, so rankings, form and Elo are about eight months old.
- Injuries, weather, altitude, fatigue within a tournament and motivation are not in the data.
- Ranking, points, seed and age come from a player's latest recorded match, so they may be stale for a match weeks later.
- Men's ATP only; women's (WTA) data would need the sister repository and small path changes in `data.py`.
- Carpet is loaded but has no dedicated flag (all three surface flags are 0). It is rare in recent seasons.
- The ranking baseline and the accuracy numbers differ from the paper because the data source differs.
- The tournament option adds history at an event but does not set the surface or format for you.

Possible extensions

- Analyse wrongly predicted matches to find missing features (the paper's main suggestion).
- Add calibration plots, SHAP explanations for tree models, or comparison with historical betting odds from tennis-data.co.uk.
- Add fatigue features (minutes played in the last week) and rest days.
- Retrain on a schedule with `--refresh` to keep the app current.

13. Glossary

Term	Meaning
Feature	A number describing a match or player that the model uses as input.

Term	Meaning
Label	The answer the model learns: 1 if Player 1 won, 0 if Player 1 lost.
Data leakage	Accidentally letting the model see information from the future, which inflates accuracy artificially.
Time-based split	Training on earlier seasons and testing on a later one, as in real forecasting.
Calibration	Whether a '70%' prediction wins about 70% of the time.
Elo rating	A running strength score updated after every match.
Imputation	Filling missing values with a sensible substitute (here, the median).
Regularisation (C)	A penalty that keeps the model from over-fitting; smaller C means stronger penalty.
Head-to-head	The record of two players against each other.
Challenger	The tier just below the main ATP tour, where many players build their ranking.

Credits

Method inspired by Ajay Krishna Amudan, *Tennis Match Prediction using Machine Learning* (Stanford CS229). Data: TML-Database (TennisMyLife), built on Jeff Sackmann / Tennis Abstract's tennis_atp data (CC BY-NC-SA 4.0).